Project Scope — 360° IFC Asset Detection, Review, and (optional) Geolocation Prototype

Client: Retail Digital Twin Platform
Date: 2025-08-27

---

0. Executive Summary

Build a prototype that automatically identifies IFC physical elements in indoor/outdoor 360° panoramas (cubemap faces), presents them for human review in a React UI, and (as a nice-to-have) geolocates assets to draw points/lines/polygons on a map. The prototype emphasizes open-source models, batch processing, and a human-in-the-loop improvement loop suitable for retail property portfolios.

---

1. Goals & Success Criteria

Business outcome: Replace manual asset identification with automated detection from property imagery.
Prototype must-haves: - Detect IFC physical elements in cubemap panoramas (all six faces). - Present detections in a React UI for user review. - Allow user to approve/decline/reclassify to improve model accuracy (labels feed the training set post-QA).

Nice-to-haves: - Use pano heading + geo to place assets on a map. - Draw asset geometries (points/lines/polygons) on a map. - Package models/services for AWS deployment.

Target accuracy: 80% confidence threshold for presenting candidates. Primary metric for V1 is **precision@0.5 IoU** at the review threshold; we will report **recall** and **per-class confusion** as secondary metrics.
Geo accuracy: No formal requirement; elevation not required.
KPI cadence: Bi-weekly reviews (demo + metrics snapshot + risks).

---

2. Scope

In-scope

- Vision models for classifying/detecting IFC physical elements in 360° cubemap panoramas.

- Review UI: panorama viewer, detection overlays, asset table, approve/reject/reclassify.
- Data pipeline & storage (S3 + PostgreSQL/PostGIS) for images, detections, assets, and reviews.
- Labeling sprint to bootstrap training data; active learning loop (post-QA).
- (Nice-to-have) Bearing-based geolocation & simple geometry extraction; map rendering in React.
- Packaging for batch inference; basic ops/observability.

Out-of-scope (for prototype)

- Full BIM reconstruction, SLAM/SfM, or accurate indoor mapping beyond bearing triangulation.
- Complex change detection/versioning across renovation phases.
- Brand-specific signage recognition (detect as generic signage).
- Enterprise SSO/RBAC (not required for prototype).

Assumptions

- Latest published full IFC version: **IFC 4.3 (IFC4x3)** as the primary mapping; fall back to IFC4 where needed.
- Panoramas captured mostly on **Ricoh Theta Z1**; cubemap faces available, with **Lat/Lon + Heading** metadata and **Level** (floor) tags; floor plans usually available.
- Pilot scale: **5 properties, 20–200 panos/property**; batch inference acceptable (e.g., nightly).
- Open-source stack; US data residency.

---

3. V1 IFC Taxonomy (Retail-oriented)

The aim is practical detection in 360s and a clean mapping to IFC4.3. Geometry type indicates how features are represented on the map in V1.

Architectural

- **IfcWall** → *line* (centerline; indoor)
- **IfcDoor** → *point*
- **IfcWindow** → *point*
- **IfcColumn** → *point* (plan location)
- **IfcRailing** → *line*
- **IfcStair / IfcStairFlight** → *polygon* (footprint) or *point* (V1 simplified)
- **IfcCovering (CEILING/FLOOR/WALLFINISH)** → *polygon* (optional; segmentation only, no map feature in V1)

MEP — HVAC & Controls

- **IfcAirTerminal** (diffuser, grille, register) → *point*
- **IfcDuctSegment** (exposed) → *line* (optional)
- **IfcUnitaryEquipment** / **IfcAirConditioningUnit** (RTU/AHU) → *point* (roof/exterior)
- **IfcDamper**/**IfcValve** (visible) → *point* (optional)
- **IfcSensor** (thermostat/temperature) → *point*

Electrical & Lighting

- **IfcLightFixture** (ceiling/wall) → *point*
- **IfcElectricalPanel** (panelboard) → *point*
- **IfcOutlet**/**IfcSwitchingDevice** (switch) → *point* (optional)
- **Emergency lighting/exit signs** → **IfcLightFixture** or **IfcSign** → *point*

Fire & Life Safety

- **IfcFireSuppressionTerminal** (sprinkler head) → *point*
- **IfcFireAlarm**/**IfcAlarm**/**IfcDetector** (smoke/heat) → *point*
- **IfcFireHydrant** (site) → *point*
- **Portable extinguisher** → **IfcFireSuppressionTerminal** → *point*

Plumbing

- **IfcSanitaryTerminal** (toilet/urinal/sink) → *point*
- **IfcWasteTerminal** (floor drain) → *point*

Site & Exterior

- **IfcPavement** (parking lot/drive) → *polygon*
- **IfcKerb** (curb) → *line*
- **IfcSidewalk** (walks) → *polygon*
- **IfcSign** (site signage/generic branded sign) → *point*
- **IfcLightingDevice** (site light pole) → *point*
- **IfcElectricChargingUnit** (EV charger) → *point*
- **IfcWasteTerminal / IfcContainer** (dumpster/corral) → *polygon* or *point*
- **IfcCoolingTower/IfcAirConditioningUnit** (rooftop) → *point*

Furnishings & Retail Fixtures (selected)

- **IfcFurniture** (bench, table) → *point*
- **IfcFurnishingElement** (shelving/gondolas) → *line* or *polygon* (optional in V1; default *point*)
- **IfcBollard** → *point*

Note: We will maintain a class→display mapping for the UI (e.g., “Door”, “Light Fixture”) and a class→IFC mapping for export. The above list can be trimmed for V1 if needed.

---

4. Data & Inputs

- **Imagery**: Cubemap (front/back/left/right/up/down) from Ricoh Theta Z1; typical 23MP pano; faces stored as JPEG/PNG; faces contain EXIF/heading when available.
- **Metadata**: `{ pano_id, property_id, level, lat, lon, heading_deg, capture_time, face_urls[] }`.
- **Floor plans**: Raster (PNG/PDF) or vector (DXF/SVG); optional alignment to world coords per site.
- **Storage**: Images in S3; metadata and results in PostgreSQL/PostGIS.

---

5. Technical Approach

5.1 Detection & Segmentation (Open-source)

- **Backbone**: Open-vocabulary detector (**GroundingDINO**) + instance segmentation (**SAM** or **HQ-SAM**) to bootstrap across many classes without huge labeled sets.

- **Fine-tuning**: After labeling sprint, fine-tune a compact detector (e.g., YOLO-family) on the curated IFC taxonomy for higher precision/latency gains.

- **360 handling**: Run inference per **cubemap face**; convert to common spherical coords for cross-face NMS/merging near seams; export masks/bboxes in both face coords and spherical coords.
- **Class prompts**: Maintain a prompt list per IFC class (and synonyms) for the open-vocabulary stage.

5.2 Tracking/Association Across Panos (optional)

- **ReID/embedding matching** to merge the same asset seen in adjacent panos (use cosine similarity on visual embeddings + spatial proximity on pano graph).

5.3 Geometry Extraction (Nice-to-have)

- **Point features**: Ray bearing from pano position using face FOV + global heading; for two or more panos that detect the same instance, **triangulate** via bearing intersections. If a single pano, place a point at pano position with bearing “direction” metadata; allow user to adjust on map.
- **Line features (walls/curbs)**: Semantic segmentation + Hough/edge tracing per face → reproject to ground plane; snap to floor plan vectors if available.

- **Polygon features (parking/sidewalk)**: Region segmentation; approximate polygon via contour simplification; allow user to edit/snap.

5.4 Quality Control & Active Learning

- **Review threshold**: Show detections ≥ 0.80 confidence.
- **QA workflow**: Reviewer confirms/rejects/reclassifies; QA lead batch-approves the reviewed set; only then labels join the training set.
- **Retraining cadence**: As needed during prototype (e.g., once after first tranche of labels).
- **Bias controls**: Sample long-tail classes for targeted labeling.

---

6. System Architecture (Prototype)

- **Frontend**: React (Vite) + MUI theme; Mapbox GL + Deck.gl; 360 pano viewer (KRPano) with cubemap support; state via Redux Toolkit or TanStack Query.
- **Backend APIs**: Node.js/Express (or FastAPI) for ingestion, auth (prototype), detections, assets, reviews, and export.
- **Batch Inference**: Python service (Docker) pulls unprocessed panos, runs CV pipeline, writes to DB/S3.
- **Storage**: S3 (images, masks), PostgreSQL/PostGIS (metadata, detections, assets, geometries).
- **Deployment**: Local Docker Compose for dev; AWS-ready containers (ECR/ECS or EC2) as a nice-to-have.

---

7. Data Model (Relational + PostGIS)

Tables (primary fields only) - `properties(id, name, addr, crs='EPSG:4326')` - `panoramas(id, property_id, level, lat, lon, heading_deg, captured_at, faces_json, storage_uri)` - `detections(id, pano_id, model_version, ifc_class, label_display, confidence, face_id, bbox_xywh, mask_uri, sphere_coords_json, created_at)` - `assets(id, property_id, ifc_class, status{proposed|confirmed|rejected}, geometry(GEOMETRYZ, SRID 4326), source_detection_ids[], attributes_json, created_at, updated_at)` - `reviews(id, detection_id, reviewer, action{confirm|reject|reclassify}, new_class, note, created_at)` - `qa_batches(id, created_at, approved_at, approver, notes)` - `training_samples(id, detection_id, asset_id, qa_batch_id, included_bool)` - `models(version, training_data_hash, metrics_json, created_at)`

Geometry conventions - Points: centroid of triangulated bearings or user-placed.
- Lines: LINESTRING in site coordinate reference; indoor wall centerlines on a per-level basis.
- Polygons: simplified, valid, closed; store area in attributes.

Exports - GeoJSON / FlatGeobuf per property, IFC-class filtered.

---

8. API Sketch (Prototype)

- `POST /ingest/pano` → register pano + faces + metadata.
- `POST /inference/run?property_id=…` → enqueue batch.
- `GET /detections?pano_id=…` → list detections with masks.
- `POST /review` → {detection_id, action, new_class?}.
- `POST /qa/approve` → approve a batch to training set.
- `GET /assets?property_id=…&status=…` → confirmed assets + geometries.
- `POST /assets/:id/geometry` → create/update geometry (point/line/polygon).
- `GET /export/geojson?property_id=…&classes=…` → GeoJSON.

---

9. Frontend (React) — User Experience

Pages/Views 1. **Pano Review** - 360 viewer with overlay for detections (color by class).
- Left panel: detections list (sortable by class/confidence), bulk actions, filters.
- Actions: confirm/reject/reclassify; keyboard shortcuts; jump-to overlay; mask toggle. 2. **Map View (Nice-to-have)** - Deck.gl layers: points (equipment), lines (walls/curbs), polygons (lots/sidewalks).
- Triangulated assets appear with uncertainty cones; editors to adjust geometry. 3. **QA & Batches** - Review history, per-batch metrics, approve-to-training. 4. **Settings (Prototype)** - IFC classes enabled, thresholds, export.

UI/Tech Stack: React + Vite, MUI (theming), Mapbox GL + Deck.gl, Marzipano (cubemap), TanStack Query. Accessibility: US English, keyboard navigable core actions.

---

10. Labeling Sprint Plan (Prototype)

- **Tool**: Open-source annotator (e.g., CVAT/Label Studio).
- **Target**: **~300–600** panos labeled (balanced indoor/outdoor, day/night), aiming for **8–15k instances** across 20–30 classes.
- **Guidelines**: Lowest reasonable class granularity; occlusions allowed if >30% visible; generic “Signage” for branded signs.
- **Review**: Daily spot-checks; weekly guideline updates.

---

11. Evaluation Plan

- **Metrics**: mAP@0.5, per-class precision/recall, review throughput (detections/min), acceptance rate.
- **Test split**: Hold-out properties (at least 1 site) never used in training.
- **Report**: Snapshot at each bi-weekly meeting; confusion matrix + top error modes.

---

12. Geolocation & Geometry (Nice-to-have)

1. **Association**: Merge same instance across neighboring panos using visual embeddings + pano graph distance.
2. **Triangulation**: Use pano pose (lat/lon/heading) + face FOV to compute bearing lines; intersect bearings from ≥2 panos; clamp to site bounds.
3. **Indoor snapping**: If a floor-plan transform exists, project features into local floor coordinates and snap lines to nearest floor-plan edges (walls/curbs).
4. **Topology rules**: Valid polygons (no self-intersection), optional snap tolerance 0.25 m; walls as simplified centerlines.

---

13. Security, Privacy, Compliance (Prototype)

- US data residency; images and outputs in US regions (if deployed to AWS).
- Face/license-plate blur respected (no unblurring).

- Minimal PII; no brand-specific signage recognition.
- Access: basic token auth; audit log of review actions.

---

14. Risks & Mitigations

- **Low light/reflective glass** → HDR/denoise pre-processing; conservative thresholds; human review.
- **Class imbalance (rare EV chargers)** → targeted sampling; class-balanced loss; data augmentation.
- **Seam artifacts in cubemap** → spherical NMS and seam padding.
- **Weak geolocation from single pano** → require ≥2-pano triangulation or manual placement; visualize uncertainty.
- **Floor plan misalignment** → manual anchor points; store transform per level.

---

15. Milestones, Deliverables, and Acceptance Criteria

M0 — Project Kickoff & Taxonomy Freeze

- Finalize V1 class list; sign off on UI sketches.
- Dev env + repositories + data schemas.
  Acceptance: Signed class list, endpoints & schema PRs merged.

M1 — Labeling Sprint & Baseline Inference

- Stand up labeling tool; label first 150–300 panos.
- Open-vocabulary baseline (GroundingDINO + SAM) producing reviewable detections.
  Acceptance: Baseline detections visible in UI; ≥15 classes with usable precision.

M2 — Review UI

- Pano review view with overlays, filters, shortcuts; review API.
- Persist review actions; simple metrics page.
  Acceptance: Reviewer can confirm/reject/reclassify; data stored; batch export works.

M3 — Fine-tuned Detector & Active Learning

- Train compact detector on labeled set; integrate model registry; run batch over pilot data.

- QA batch approval gates labels into training set.
  Acceptance: ≥0.60 mAP@0.5 overall on hold-out; precision ≥0.75 for top 10 classes at 0.8 threshold (targets adjustable).

M4 — Map & Geometry (Nice-to-have)

- Bearing triangulation for points; optional snapping to floor plan.

- Deck.gl layers for points/lines/polys; geometry editor (move vertex, add vertex).
  Acceptance: User can place/adjust features on map; export GeoJSON.

M5 — Packaging & Handoff

- Dockerized services; seed dataset; admin docs & runbooks; AWS-ready images.
  Acceptance: One-command local deploy; docs enable your team to run batch inference and review.

Bi-weekly demos: End of Weeks 2, 4, 6, 8.

---

16. Acceptance Tests (Must-haves)

1. Upload a property (≥50 panos) → run batch inference → detections appear in UI within the next scheduled batch.

2. Reviewer confirms/rejects ≥100 detections across ≥10 classes; reclassification works; actions are persisted and auditable.

3. QA approves batch; export creates a labeled dataset; retrain job consumes it (simulated or actual).

4. (Optional) At least one site shows triangulated rooftop HVAC and indoor light fixtures on the map; user can adjust points and export GeoJSON.
